2 Roteiro 02 – Programação Procedural vs Orientada a Objetos em Python

Objetivos

- Compreender a diferença entre programação procedural e orientada a objetos;
- Implementar um mesmo problema utilizando dois paradigmas;
- Identificar vantagens organizacionais da POO;
- Desenvolver pensamento estruturado na modelagem de sistemas.

Ferramentas Necessárias

Python 3.10+, VSCode (ou IDE similar), terminal Linux, Git.

Fundamentação Teórica

Existem diferentes paradigmas de programação. Dois dos mais utilizados são:

Programação Procedural

- Organiza o código em funções;
- Trabalha diretamente com estruturas de dados;
- Separação entre dados e comportamentos;
- Adequada para programas pequenos.

Programação Orientada a Objetos (POO)

- Organiza o código em classes;
- Dados e comportamentos ficam juntos;
- Permite maior modularização;
- Facilita manutenção e escalabilidade.

Comparação Procedural vs POO

Aspecto	**Programação Procedural**	**Programação Orientada a Objetos**
Organização do Código	Funções separadas dos dados	Dados e comportamentos organizados dentro da classe
Estrutura dos Dados	Dicionários e variáveis soltas	Objetos estruturados
Reutilização	Limitada, depende de funções isoladas	Alta, por meio de classes e métodos
Escalabilidade	Pode se tornar desorganizada em sistemas grandes	Mais adequada para sistemas complexos
Manutenção	Alterações exigem modificar várias funções	Mudanças podem ser concentradas na classe
Legibilidade	Boa em programas pequenos	Melhor organização em projetos maiores
Encapsulamento	Não há encapsulamento real	Permite encapsular dados e regras

Nesta aula, resolveremos o mesmo problema utilizando as duas abordagens.

Problema proposto: criar um pequeno sistema de cadastro de alunos.

Parte 1 – Implementação Procedural

Nesta primeira etapa, resolveremos o problema sem utilizar classes.

Atividade 1 — Criando Estrutura Inicial

Criar a pasta do projeto:

```
mkdir sistema-alunos
cd sistema-alunos
touch procedural.py
git init
```

Atividade 2 — Implementando Versão Procedural

No arquivo `procedural.py`, escrever:

```
def apresentar(nome, idade):
    return f"Olá, meu nome é {nome} e tenho {idade} anos."

def maior_de_idade(idade):
    return idade >= 18

alunos = [
    {"nome": "João", "idade": 17},
    {"nome": "Maria", "idade": 16},
    {"nome": "Carlos", "idade": 18}
]

for aluno in alunos:
    print(apresentar(aluno["nome"], aluno["idade"]))

    if maior_de_idade(aluno["idade"]):
        print(f"{aluno['nome']} é maior de idade.")
```

Executar:

```
python procedural.py
```

Micro-Commit 1 — Versão Procedural

```
git add .
git commit -m "Implementa versão procedural do sistema de alunos"
```

Análise da Implementação Procedural

Observe que:

- Os dados dos alunos estão separados das funções;
- Precisamos passar manualmente os valores para cada função;
- Conforme o sistema cresce, a organização tende a ficar mais complexa.

Agora vamos resolver o mesmo problema utilizando Programação Orientada a Objetos.

Parte 2 – Implementação Orientada a Objetos

Nesta etapa, utilizaremos classes para organizar melhor o sistema.

Atividade 3 — Criando Estrutura para POO

Criar novos arquivos:

```
touch aluno.py
touch main.py
```

Atividade 4 — Criando a Classe Aluno

No arquivo `aluno.py`, escrever:

```
class Aluno:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def apresentar(self):
        return f"Olá, meu nome é {self.nome} e tenho {self.idade} anos."

    def maior_de_idade(self):
        return self.idade >= 18
```

Atividade 5 — Instanciando Objetos

No arquivo `main.py`, escrever:

```
from aluno import Aluno

alunos = [
    Aluno("João", 17),
    Aluno("Maria", 16),
    Aluno("Carlos", 18)
]

for aluno in alunos:
    print(aluno.apresentar())

    if aluno.maior_de_idade():
        print(f"{aluno.nome} é maior de idade.")
```

Executar:

```
python main.py
```

Micro-Commit 2 — Versão Orientada a Objetos

```
git add .
git commit -m "Implementa versão orientada a objetos do sistema"
```

Comparação Final

Após implementar as duas versões, reflita:

- Em qual modelo o código ficou mais organizado?
- Qual abordagem facilita adicionar novas funcionalidades?
- Qual modelo seria mais sustentável para um sistema maior?

A Programação Orientada a Objetos tende a se tornar mais vantajosa conforme a complexidade do sistema aumenta.

Evolução do Sistema — Aumentando a Complexidade

Agora vamos evoluir o sistema. Além de nome e idade, cada aluno deverá possuir:

- Lista de notas;
- Cálculo de média;
- Verificação de aprovação (média >= 7);
- Possibilidade de adicionar notas dinamicamente.

Primeiro faremos isso de forma procedural.

Atividade 6 — Evoluindo a Versão Procedural

Nesta etapa, iremos modificar o arquivo `procedural.py` para incluir notas e cálculo de média. Substituir o código anterior por:

```
def apresentar(nome, idade):
    return f"{nome} ({idade} anos)"

def calcular_media(notas):
    if len(notas) == 0:
        return 0
    return sum(notas) / len(notas)

def verificar_aprovacao(media):
    return media >= 7

def adicionar_nota(aluno, nota):
    aluno["notas"].append(nota)

alunos = [
    {"nome": "João", "idade": 17, "notas": [6, 7, 8]},
    {"nome": "Maria", "idade": 16, "notas": [5, 6, 7]},
    {"nome": "Carlos", "idade": 18, "notas": [8, 9, 10]}
]

for aluno in alunos:
    print(apresentar(aluno["nome"], aluno["idade"]))

    media = calcular_media(aluno["notas"])
    print("Média:", media)

    if verificar_aprovacao(media):
        print("Aprovado")
    else:
        print("Reprovado")

    print("-" * 20)
```

Executar:

```
python procedural.py
```

Micro-Commit 3 — Procedural Evoluído

```
git add .
git commit -m "Evolui versão procedural com notas e média"
```

Análise Crítica da Versão Procedural

Observe atentamente:

- As funções dependem da estrutura do dicionário;
- Se alterarmos o formato do aluno, várias funções precisam ser modificadas;
- A lógica de negócio está espalhada;
- Não há proteção dos dados internos.

Agora vamos resolver o mesmo problema utilizando Programação Orientada a Objetos.

Atividade 7 — Evoluindo a Classe Aluno

Modificar o arquivo `aluno.py` para:

```
class Aluno:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
        self.notas = []

    def adicionar_nota(self, nota):
        self.notas.append(nota)

    def calcular_media(self):
        if len(self.notas) == 0:
            return 0
        return sum(self.notas) / len(self.notas)

    def aprovado(self):
        return self.calcular_media() >= 7

    def apresentar(self):
        return f"{self.nome} ({self.idade} anos)"
```

Atividade 8 — Atualizando o main.py

Modificar o arquivo `main.py` para:

```
from aluno import Aluno

alunos = [
    Aluno("João", 17),
    Aluno("Maria", 16),
    Aluno("Carlos", 18)
]

alunos[0].adicionar_nota(6)
alunos[0].adicionar_nota(7)
alunos[0].adicionar_nota(8)

alunos[1].adicionar_nota(5)
alunos[1].adicionar_nota(6)
alunos[1].adicionar_nota(7)

alunos[2].adicionar_nota(8)
alunos[2].adicionar_nota(9)
alunos[2].adicionar_nota(10)

for aluno in alunos:
    print(aluno.apresentar())
    print("Média:", aluno.calcular_media())

    if aluno.aprovado():
        print("Aprovado")
    else:
        print("Reprovado")

    print("-" * 20)
```

Executar:

```
python main.py
```

Micro-Commit 4 — POO Evoluída

```
git add .
git commit -m "Evolui classe Aluno com notas e cálculo de média"
```

Reflexão Final

Compare as duas abordagens e responda:

- Qual versão ficou mais organizada?
- Em qual modelo seria mais fácil adicionar novas regras?
- Se o sistema tivesse 1000 alunos, qual modelo seria mais sustentável?

A Programação Orientada a Objetos tende a se tornar mais vantajosa conforme a complexidade do sistema aumenta.

Entrega

O estudante deverá:

- Implementar as duas versões do sistema;
- Organizar o código corretamente;
- Realizar commits descritivos;
- Criar repositório público no GitHub;
- Enviar o link do repositório.

O projeto deverá estar funcional.